

Цель Блокнота

Рассмотрение механизмов интерпритации моделей машинного обучения в рамках библиотеки `lime`

Import библиотек

Обновим `scikit-learn` чтобы воспользоваться некими новинками

Например `TargetEncoder`

```
!pip install -U scikit-learn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (1.2.2)
Collecting scikit-learn
  Downloading scikit_learn-1.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (13.3 MB)
    13.3/13.3 MB 36.5 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.25.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.11.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (3.5.0)
Installing collected packages: scikit-learn
  Attempting uninstall: scikit-learn
    Found existing installation: scikit-learn 1.2.2
    Uninstalling scikit-learn-1.2.2:
      Successfully uninstalled scikit-learn-1.2.2
Successfully installed scikit-learn-1.5.0
```

```
!pip install catboost
```

```
Collecting catboost
  Downloading catboost-1.2.5-cp310-cp310-manylinux2014_x86_64.whl (98.2 MB)
    98.2/98.2 MB 5.5 MB/s eta 0:00:00
Requirement already satisfied: graphviz in /usr/local/lib/python3.10/dist-packages (from catboost) (0.20.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from catboost) (3.7.1)
Requirement already satisfied: numpy>=1.16.0 in /usr/local/lib/python3.10/dist-packages (from catboost) (1.25.2)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.10/dist-packages (from catboost) (2.0.3)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from catboost) (1.11.4)
Requirement already satisfied: plotly in /usr/local/lib/python3.10/dist-packages (from catboost) (5.15.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from catboost) (1.16.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24->catboost) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24->catboost) (2023.4)
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24->catboost) (2024.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (1.2.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (4.51.0)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (24.0)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (9.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (3.1.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from plotly->catboost) (8.3.0)
Installing collected packages: catboost
Successfully installed catboost-1.2.5
```

Гигантский блок импорта

[Показать код](#)

Набор данных

Набор данных представляет собой статистику параметров автомобилей на вторичном рынке в Молдавии.

Набор включает ряд категориальных и численных значений, составляющих одну запись (строку). Число записей можно найти как число строк.

Каждый столбец в записи — это отдельный параметр.

Среди указанных параметров приведены целевой для задачи предсказания (регрессии) - цена автомобиля.

Также среди параметров есть целевой для задачи классификации - тип трансмиссии.

Последняя задача может быть рассмотрена, например, как пример задачи на заполнение пропусков (если продавец не указал соответствующий параметр).

Загрузка данных

Загружаем наши данные из файла по прямой ссылке на git-hub



```
DF = pd.read_csv('https://raw.githubusercontent.com/dayekb/Basic_ML_Alg/main/cars_moldova_clean.csv', delimiter = ',')
DF
```

	Make	Model	Year	Style	Distance	Engine_capacity(cm3)	Fuel_type	Transmission	Price(euro)
0	Toyota	Prius	2011	Hatchback	195000.0	1800.0	Hybrid	Automatic	7750.0
1	Renault	Grand Scenic	2014	Universal	135000.0	1500.0	Diesel	Manual	8550.0
2	Renault	Laguna	2012	Universal	110000.0	1500.0	Diesel	Manual	6550.0
3	Opel	Astra	2006	Universal	200000.0	1600.0	Metan/Propan	Manual	4100.0
4	Mercedes	Vito	2000	Microvan	300000.0	2200.0	Diesel	Manual	3490.0
...	...	...	...	...	...	...	...	...	...
32480	Volkswagen	Passat	2016	Sedan	88000.0	1800.0	Petrol	Automatic	11500.0
32481	Land Rover	Freelander	2002	Crossover	225000.0	1800.0	Metan/Propan	Manual	4400.0
32482	Dacia	Logan Mcv	2015	Universal	89000.0	1500.0	Diesel	Manual	7000.0
32483	Mazda	6	2006	Combi	370000.0	2000.0	Diesel	Manual	4000.0
32484	Renault	Grand Scenic	2006	Minivan	300000.0	1500.0	Diesel	Manual	4000.0

32485 rows x 9 columns

```
# Добавим пару случайных признаков для последующего анализа
DF['random_feature_1'] = np.random.random(size = DF.shape[0]) # равномерное распределение
DF['random_feature_2'] = np.random.normal(size = DF.shape[0]) # нормальное распределение
```

Список числовых и категориальных данных

```
cat_columns = ['Make', 'Model', 'Style', 'Fuel_type', 'Transmission']
num_columns = ['Year', 'Distance', 'Engine_capacity(cm3)', 'Price(euro)']
```

Предварительная обработка из [предыдущего блокнота](#)

Из предварительной обработки исключена колонка `Price(euro)`, поскольку она является целевой меткой и будет обработана отдельно

> Всякие там Pipeline

[Показать код](#)

> Вспомогательные функции

[Показать код](#)

```
scoring_reg = {'R2': 'r2',
               '-MSE': 'neg_mean_squared_error',
               '-MAE': 'neg_mean_absolute_error',
               '-Max': 'max_error'}

cv_rule = ShuffleSplit(n_splits=5, random_state = 42)

metrics = [mse, r2_score]

metric_names = ['mse', 'r2_score']
```

✓ Линейная Регрессия

Считываем данные, разбиваем на тестовую и тренировочную

```
# не забываем удалить целевую переменную цену из признаков
X,y = DF.drop(columns = ['Price(euro)'], DF['Price(euro)'])

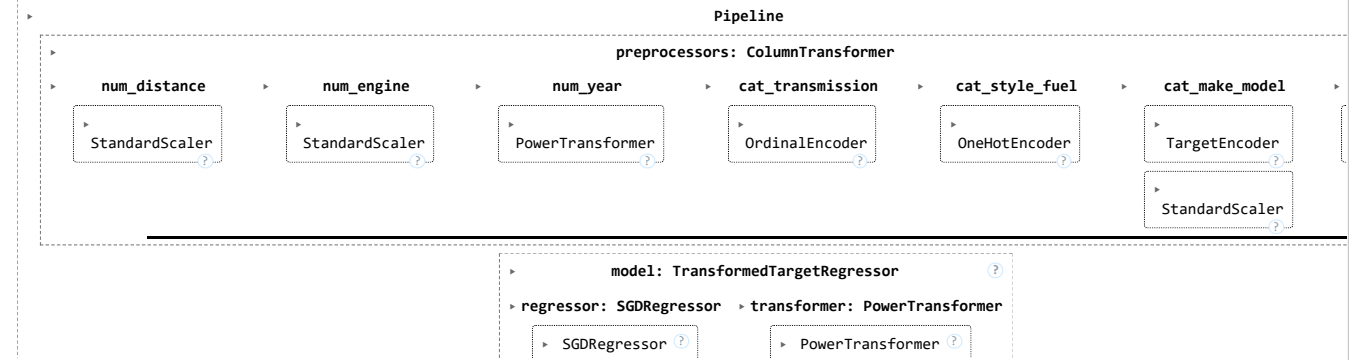
# разбиваем на тестовую и валидационную
X_train, X_val, y_train, y_val = train_test_split(X, y,
                                                  test_size=0.3,
                                                  random_state=42)
```

Запускаем Pipeline из предварительной обработки и модели

```
pipe_all_transform = Pipeline([
    ('preprocessors', preprocessors_all),
    ('model', TransformedTargetRegressor( regressor=SGDRegressor(random_state = 42),
    transformer=PowerTransformer())
    )
])
```

```
%%time
pipe_all_transform.fit(X_train, y_train)
```

CPU times: user 233 ms, sys: 12.5 ms, total: 245 ms
Wall time: 251 ms



✓ CatBoost

```
train_data = Pool(
    data=X_train,
    label=y_train,
    cat_features = cat_columns,
    feature_names=list(X_train.columns)
)

eval_data = Pool(
    data=X_val,
    label=y_val,
    cat_features = cat_columns,
    feature_names=list(X_train.columns)
)
```

```
model_reg = CatBoostRegressor(iterations = 1000,
                              early_stopping_rounds=100,
                              verbose = 100,
                              cat_features = cat_columns,
                              depth = 3,
                              eval_metric= 'MAE'
                              )
```

```
%%time
model_reg.fit(X=train_data,
              eval_set=eval_data,
              );
```

```
Learning rate set to 0.083159
0:   learn: 5341.6428495   test: 5427.6644559   best: 5427.6644559 (0) total: 73.6ms remaining: 1m 13s
100: learn: 2049.4252973   test: 2073.1290595   best: 2073.1290595 (100) total: 1.24s remaining: 11s
200: learn: 1921.0906697   test: 1953.4236166   best: 1953.4236166 (200) total: 2.37s remaining: 9.41s
300: learn: 1872.9839472   test: 1910.0682918   best: 1909.7887651 (299) total: 3.27s remaining: 7.59s
400: learn: 1837.2600393   test: 1881.4771771   best: 1881.4771771 (400) total: 5.28s remaining: 7.89s
500: learn: 1807.5440048   test: 1860.0624414   best: 1860.0624414 (500) total: 8.8s remaining: 8.77s
600: learn: 1788.1609320   test: 1846.4899214   best: 1846.4899214 (600) total: 11.2s remaining: 7.45s
700: learn: 1768.6095999   test: 1833.3200671   best: 1833.3200671 (700) total: 13.6s remaining: 5.81s
800: learn: 1751.9891693   test: 1821.9861518   best: 1821.9861518 (800) total: 16s remaining: 3.97s
900: learn: 1737.9662836   test: 1813.3286209   best: 1813.2794116 (897) total: 18.6s remaining: 2.05s
999: learn: 1724.4595832   test: 1803.3583870   best: 1803.3583870 (999) total: 22.3s remaining: 0us
```

```
bestTest = 1803.358387
bestIteration = 999
```

```
CPU times: user 18.6 s, sys: 1.6 s, total: 20.2 s
Wall time: 22.8 s
<catboost.core.CatBoostRegressor at 0x7a8204d40c10>
```

✓ Lime

Надо устанавливать библиотеку lime - но тут стандартно

```
!pip install lime
```

```
Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
    275.7/275.7 kB 3.4 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from lime) (3.7.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from lime) (1.25.2)
```



```
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from lime) (1.11.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from lime) (4.66.4)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.10/dist-packages (from lime) (1.5.0)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.10/dist-packages (from lime) (0.19.3)
Requirement already satisfied: networkx>=2.2 in /usr/local/lib/python3.10/dist-packages (from scikit-image>=0.12->lime) (3.3)
Requirement already satisfied: pillow!=7.1.0,!>7.1.1,!>8.3.0,>=6.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-image>=0.12->lime) (9.4.0)
Requirement already satisfied: imageio>=2.4.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image>=0.12->lime) (2.31.6)
Requirement already satisfied: tifffile>=2019.7.26 in /usr/local/lib/python3.10/dist-packages (from scikit-image>=0.12->lime) (2024.5.22)
Requirement already satisfied: PyWavelets>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image>=0.12->lime) (1.6.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from scikit-image>=0.12->lime) (24.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.18->lime) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.18->lime) (3.5.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.5.0->lime) (1.2.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.5.0->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.5.0->lime) (4.51.0)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.5.0->lime) (1.4.5)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.5.0->lime) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.5.0->lime) (2.8.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.7->matplotlib>=3.5.0->lime) (1.16.0)
Building wheels for collected packages: lime
Building wheel for lime (setup.py) ... done
Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283835 sha256=2382eedb50700950cb866c9403dc085db6515a6aa0b5cea2804faa9d96b2758
Stored in directory: /root/.cache/pip/wheels/fd/a2/af/9ac0a1a85a27f314a06b39e1f492bee1547d52549a4606ed89
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
```

```
import lime
import lime.lime_tabular
```

Для линейной регрессии

Опять в pipeline не встраивается

Надо преобразовывать

```
data_transform = pipe_all_transform['preprocessors'].transform(X_val)
feature_names=pipe_all_transform['preprocessors'].get_feature_names_out()
```

Интерфейс похож на то что в shap

Создаем объект класса Explainer

```
explainer = lime.lime_tabular.LimeTabularExplainer(data_transform,
                                                    feature_names=feature_names,
                                                    class_names=['price'],
                                                    verbose=True,
                                                    mode='regression')
```

Тут с выходом скучно

Только локальная интерпретируемость

```
#@title { run: "auto" }
index = 4030 #@param {type:"slider", min:0, max:9746, step:10}

exp = explainer.explain_instance(data_transform[index],
                                pipe_all_transform['model'].regressor._p,
                                num_features=10)

exp.show_in_notebook(show_table=True)
```

index

4030

[Скрыть код](#)

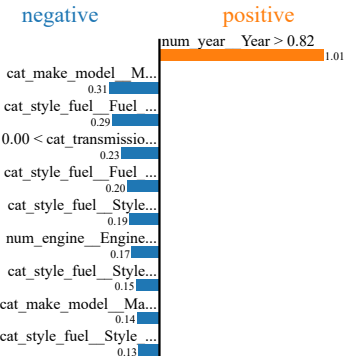
Intercept 1.0149564530885697
Prediction_local [0.21298298]
Right: 0.7243586321853795

Predicted value

-2.12 (min)

0.72

2.00 (max)



Feature	Value
num_year_Year	2.09
cat_make_model_Model	-0.79
cat_style_fuel_Fuel_type_Electric	0.00
cat_transmission_Transmission	1.00
cat_style_fuel_Fuel_type_Plug-in Hybrid	0.00
cat_style_fuel_Style_Pickup	0.00
num_engine_Engine_capacity(cm3)	-0.85
cat_style_fuel_Style_Roadster	0.00
cat_make_model_Make	-1.19
cat_style_fuel_Style_Cabriolet	0.00

Ну или текстом



exp.as_list()

```
[('num_year__Year > 0.82', 1.0051531295255638),
 ('cat_make_model__Model <= -0.66', -0.3081922723641998),
 ('cat_style_fuel__Fuel_type_Electric <= 0.00', -0.2917513225119983),
 ('0.00 < cat_transmission__Transmission <= 1.00', -0.23485406617357385),
 ('cat_style_fuel__Fuel_type_Plug-in Hybrid <= 0.00', -0.20015133600377016),
 ('cat_style_fuel__Style_Pickup <= 0.00', -0.18708831125447495),
 ('num_engine__Engine_capacity(cm3) <= -0.56', -0.17127792328917202),
 ('cat_style_fuel__Style_Roadster <= 0.00', -0.14520291077077635),
 ('cat_make_model__Make <= -0.67', -0.14018211010834059),
 ('cat_style_fuel__Style_Cabriolet <= 0.00', -0.12842635330355673)]
```

▼ Для CatBoost

И тут снова боль

Просто так не получится

Нужно создать новый класс в котором всё будет "хорошо"

```
#на вход поступает модель, список колонок и список категориальных значений
class catboost_predict_proba_wrapper():
    def __init__(self,model,column_names, list_cat_feat):
        self.model = model
        self.list_cat_feat = list_cat_feat
        self.column_names = column_names
    # жизненно важно реализовать метод predict
    # в котором для CatBoost мы реализуем перевод данных в Pool
    def predict(self,this_array):
        shape_tuple = np.shape(this_array)
        if len(shape_tuple) == 1:
            this_array = this_array.reshape(1,-1)

        pandas_df = pd.DataFrame(data=this_array,columns=self.column_names)

        pandas_df[X_val.columns[list_cat_feat]] = pandas_df[X_val.columns[list_cat_feat]].astype(str)

        self.pandas_df = pandas_df
        self.data_pool = Pool(self.pandas_df,cat_features=self.list_cat_feat)
        self.predictions = self.model.predict(self.data_pool)
        return self.predictions
```

```
list_cat_feat = []
for col in cat_columns:
    list_cat_feat.append(list(X_val.columns).index(col))
```

Оно плохо работает с категориальными признаками, которые слова

Поэтому надо предварительно обработать

Числовые признаки - с ними ничего не делаем

Категориальные - просто переводим в цифры (CatBoost разберется)

```
cat_pipe_ = Pipeline([
    ('encoder', LabelEncoder())

])

preprocessors_cb = ColumnTransformer(transformers=[
    ('columns_cat', cat_pipe_, cat_columns),
    ('columns_num', "passthrough", columns_num),
])
```

```
for col in cat_columns:
    le = LabelEncoder()
    le.fit(X_val[col].values)
    X_val[col] = le.transform(X_val[col].values)
```

X_val



	Make	Model	Year	Style	Distance	Engine_capacity(cm3)	Fuel_type	Transmission	random_feature_1	random_feature_2
3835	32	176	2011	3	46000.0	4800.0	4	0	0.048944	-0.250780
3578	15	157	2011	3	106000.0	2200.0	0	0	0.214187	0.588193
17679	2	86	2010	4	165000.0	1400.0	4	0	0.936424	-0.033228
31387	3	42	1991	10	30000.0	1600.0	4	1	0.199056	-0.205835
12801	25	221	2010	10	90000.0	2200.0	0	0	0.161343	0.024457
...	...	...	...	...	...	...	...	...	...	...
29998	43	465	2006	3	125000.0	2200.0	0	1	0.390957	-1.591457
23366	43	121	2004	6	265700.0	2000.0	0	1	0.013863	-0.070270
23503	15	96	2009	10	94000.0	2362.0	4	0	0.990566	0.317596
15046	3	588	2004	3	280000.0	4400.0	4	0	0.201962	0.858872
31226	29	318	2013	3	95000.0	1600.0	4	0	0.381302	0.336190

9746 rows × 10 columns

feature_names

```
array(['num_distance__Distance', 'num_engine__Engine_capacity(cm3)',  
      'num_year__Year', 'cat_transmission__Transmission',  
      'cat_style_fuel__Style_Cabriolet', 'cat_style_fuel__Style_Combi',  
      'cat_style_fuel__Style_Coupe', 'cat_style_fuel__Style_Crossover',  
      'cat_style_fuel__Style_Hatchback',  
      'cat_style_fuel__Style_Microvan', 'cat_style_fuel__Style_Minivan',  
      'cat_style_fuel__Style_Pickup', 'cat_style_fuel__Style_Roadster',  
      'cat_style_fuel__Style_SUV', 'cat_style_fuel__Style_Sedan',  
      'cat_style_fuel__Style_Universal',  
      'cat_style_fuel__Fuel_type_Diesel',  
      'cat_style_fuel__Fuel_type_Electric',  
      'cat_style_fuel__Fuel_type_Hybrid',  
      'cat_style_fuel__Fuel_type_Metan/Propan',  
      'cat_style_fuel__Fuel_type_Petrol',  
      'cat_style_fuel__Fuel_type_Plug-in Hybrid', 'cat_make_model__Make',  
      'cat_make_model__Model', 'random__random_feature_1',  
      'random__random_feature_2'], dtype=object)
```

```
explainer = lime.lime_tabular.LimeTabularExplainer(X_val.values,  
                                                    class_names=['price'],  
                                                    feature_names= X_val.columns,  
                                                    categorical_features=list_cat_feat,  
                                                    categorical_names=cat_columns,  
                                                    mode='regression'  
                                                    )
```

X_val.columns[list_cat_feat]

Index(['Make', 'Model', 'Style', 'Fuel_type', 'Transmission'], dtype='object')

cat_wrapper = catboost_predict_proba_wrapper(model_reg,X_val.columns,list_cat_feat)

```
##@title { run: "auto" }  
index = 4030 ##@param {type:"slider", min:0, max:9746, step:10}
```

index

4030

```
exp = explainer.explain_instance(X_val.iloc[index].values.ravel(),  
                                cat_wrapper.predict,num_features=10)  
exp.show_in_notebook(show_all=True,  
                     show_predicted_value=True,  
                     show_table=True)
```

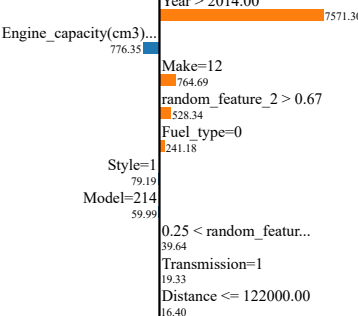
[Скрыть код](#)

Predicted value

4655.58 (min) 20231.97 (max) 33336.25 (max)

negative

positive



Feature

Value

Make=12	True
Model=214	True
Year	2021.00
Style=1	True
Distance	5.00
Engine_capacity(cm3)	1300.00
Fuel_type=0	True
Transmission=1	True
random_feature_1	0.40
random_feature_2	0.96

